Experiment No. 23

# Alpha-Beta Pruning Algorithm for Gaming (AI)

## Aim

To implement the Alpha-Beta Pruning algorithm to optimize the Minimax search by eliminating branches that cannot influence the final decision.

## Objectives

- To understand how Alpha-Beta pruning improves upon plain Minimax.
- To maintain alpha (best Maximizer value) and beta (best Minimizer value) bounds.
- To prune branches where beta <= alpha.
- To obtain the same optimal result as Minimax with fewer evaluations.

## Algorithm

1. Start.
2. Represent the game as a tree with leaf nodes holding utility values.
3. Initialize alpha = -infinity and beta = +infinity at the root.
4. If the current node is a leaf, return its value.
5. For a Maximizer node, update alpha with the best value found so far.
6. For a Minimizer node, update beta with the best value found so far.
7. If at any point beta <= alpha, prune (skip) the remaining branches.
8. Recursively evaluate all non-pruned nodes from leaves to root.
9. Display the optimal value and the number of branches pruned.
10. Stop.

## Source Code (Python)

```
tree = [3, 5, 6, 9, 1, 2, 0, -1]
pruned_count = 0
def alphabeta(depth, node_index, is_max, values, alpha, beta, max_depth):
    global pruned_count
    if depth == max_depth:
        return values[node_index]
    if is_max:
        best = float('-inf')
        for i in range(2):
            val = alphabeta(depth+1, node_index*2+i, False, values, alpha, beta, max_depth)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                pruned_count += 1
                break
        return best
    else:
        best = float('inf')
        for i in range(2):
            val = alphabeta(depth+1, node_index*2+i, True, values, alpha, beta, max_depth)
            best = min(best, val)
```

```
            beta = min(beta, best)
            if beta <= alpha:
                pruned_count += 1
                break
        return best
import math
max_depth = int(math.log2(len(tree)))
print("Leaf node values (game tree):", tree)
print(f"Tree depth: {max_depth}\n")
optimal_value = alphabeta(0, 0, True, tree, float('-inf'), float('inf'), max_depth)
print(f"The optimal value for the Maximizer is: {optimal_value}")
print(f"Number of branches pruned: {pruned_count}")
```

## Output

```
Leaf node values (game tree): [3, 5, 6, 9, 1, 2, 0, -1]
Tree depth: 3
The optimal value for the Maximizer is: 5
Number of branches pruned: 2
```

## Result

The Alpha-Beta Pruning algorithm was successfully implemented. The optimal value for the Maximizer was found to be 5, with 2 branches pruned compared to plain Minimax.